

[External] Tracer Darts Manual v1.4.0

One-time setup	2
Set up python virtual environment	2
Prepare the required apps	2
Everytime before you run Tracer Darts	3
Test Documentation	4
Result Files	4
Run all tests	4
Run several tests at once	4
Perfetto Trace	5
Common Flags	5
subatomic_app_start	6
subatomic_app_to_app	7
subatomic_dreamx_from_launcher	8
subatomic_katniss_all_during_playback	8
subatomic_katniss_card_during_playback	9
subatomic_katniss_from_launcher	9
subatomic_launcher_after_playback	10
subatomic_memory_breakdown	13
subatomic_memory_stats - app switching	15
subatomic_memory_stats - playback	16
subatomic_settings_during_playback	17
subatomic_settings_from_launcher	18
Appendix: Mobile VPN hotspot for TV	18

One-time setup

Set up python virtual environment

```
Python
# Python version should be python3.11 or newer
$ python3 --version

# Create virtual environment
$ python3 -m venv ~/tracer-darts-env

# Activate the virtual environment
$ source ~/tracer-darts-env/bin/activate

# Install packages with requirment.txt
(tracer-darts-env) $ python3 -m pip install -r requirements.txt
```

Prepare the required apps

Prerequisites: If you are not in the US, please VPN to the US.

TracerDarts should auto-install the missing apps. But if auto-install fails, you can open the apps on the playstore with the following command and install them manually. After that, please login to **HBO max** and **Netflix**.

```
Python
# Amazon Prime
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.amazon.amazonvideo.livingroom"

# AppleTV
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.apple.atve.androidtv.appletv"

# Disney+
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.disney.disneyplus"

# Hulu
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.hulu.livingroomplus"
```

```
# HBO max (need login for test)
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.wbd.stream"

# Netflix (need login for test)
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.netflix.ninja"

# Paramount+
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.cbs.ott"

# Pluto TV
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=tv.pluto.android"

# Youtube
adb shell "am start -a android.intent.action.VIEW -d \
market://details?id=com.google.android.youtube.tv"
```

Everytime before you run Tracer Darts

1. If you are not in the US, VPN to US following [Appendix: Mobile VPN hotspot for TV](#). Do **NOT** directly connect VPN with your TV (e.g. using NorVPN from TV), this could affect the TV performance.

2. Activate the virtual environment:

Python

```
# Activate the virtual environment
$ source ~/tracer-darts-env/bin/activate
```

3. Ensure only one device is connected through ADB. If multiple devices are connected to the host machine, use the **ANDROID_SERIAL** environment variable to select one ADB device to use in this shell.

Python

```
# Check your device serial
$ adb devices
DEVICE_SERIAL1 device
```

```
DEVICE_SERIAL2 device
```

```
# Select the device to run Tracer Darts  
$ export ANDROID_SERIAL=DEVICE_SERIAL2
```

Test Documentation

Result Files

After each run of Tracer Darts, the results are aggregated into the result directory **csvs/**. This directory contains the following:

1. **csvs/raw/**: The raw CSVs for each test.
2. **csvs/summary/**: The summary CSV files containing aggregated results to make performance comparisons easier.
3. **csvs/unified.csv**: The CSV file containing all metrics collected during a test run. This is to be used with the performance result template to evaluate results.

Run all tests

```
Python  
# -all to run all test. Takes about 24-30 hours  
python3 run_subatomic.py -all -o output_all
```

Run several tests at once

```
Python  
# Runs several tests at once. This test runs:  
#   -ata for subatomic_app_to_app test  
#   -dfl for subatomic_dreamx_from_launcher  
#   -qas for subatomic_app_start test  
python3 run_subatomic.py -ata -dfl -qas -o output_mixed_test
```

Perfetto Trace

Perfetto tracing should **NOT** be enabled when collecting performance benchmarks as it will negatively affect performance results.

Python

```
# -t to capture the trace along the test.  
# Takes longer time. Costs around 30MB to device system memory.  
python3 run_subatomic.py -all -t -o output_all
```

Perfetto is a performance tool. See [Perfetto](#).

If you specify the test with **-t**, you will see **.trace** files under **traces/** folder. Open those files with [Perfetto UI](#). See an example perfetto trace [here](#).

Common Flags

- **-c**
Force LauncherX to cold start during the Playback to Launcher test. This measures both [TTID](#) and [TTFD](#).
- **-i**
Number of iterations to run for each test.
- **-m**
Applies memory pressure to the device to simulate less available memory. For example, **-m 100** will simulate the device with 100 MB less RAM available.
- **-l**
Specifies playback during the Playback to Launcher test. Allows debugging issues only seen during long duration playback.
- **-disk_io**
Measure disk performance during the Playback to Launcher test. Also analyzes the launcher start time by resource delay: waiting on disk, waiting on cpu, etc.

subatomic_app_start

Measures start time of apps.

The app start time is measured with [Time to Initial Display](#) (TTID) by ActivityManager. This is the main metric collected by Scorecard and Pitot to measure app start time. This test cycles through each app, opening each one in succession.

This test can also help debug issues with the Low Memory Killer, as a too lenient LMK will noticeably worsen these metrics.

Python

```
# -qas to run subatomic_app_start test.  
python3 run_subatomic.py -qas -o output_app_start
```

name	unit	description
amazon_from_launcher-start_time	ms	Measures Amazon start time from launcher
disney_from_launcher-start_time	ms	Measures Disney start time from launcher
pluto_from_launcher-start_time	ms	Measures Pluto start time from launcher
hulu_from_launcher-start_time	ms	Measures Hulu start time from launcher
hbo_from_launcher-start_time	ms	Measures Max start time from launcher
appletv_from_launcher-start_time	ms	Measures Apple start time from launcher
netflix_from_launcher-start_time	ms	Measures Netflix start time from launcher
youtube_from_launcher-start_time	ms	Measures Youtube start time from launcher

subatomic_app_to_app

Measures app start latency during video playback.

This test plays a video and starts another app during the playback. The video is played at the device's maximum resolution to adequately stress the system.

This CUJ simulates a user pressing a remote button to open a new app during playback. For example, pressing the Youtube button on the remote during Netflix playback.

This test represents the hardest common CUJ on TV.

Python

```
# -ata to run subatomic_app_to_app test.  
python3 run_subatomic.py -ata -o output_app_to_app
```

name	unit	description
netflix_from_youtube-start_time	ms	Measures Netflix start time during Youtube playback
launcher_from_netflix_from_youtube-start_time	ms	Measures launcher start time after app to app usage
youtube_from_netflix-start_time	ms	Measures Youtube start time during Netflix playback
launcher_from_youtube_from_netflix-start_time	ms	Measures launcher start time after app to app usage

subatomic_dreamx_from_launcher

Measures Ambient Mode start time from the launcher.

This test plays a video to evict the memory of the launcher and Ambient Mode. This is because cycling between Launcher and Ambient each iteration does not represent the real use case. It then goes to the launcher and invokes Ambient Mode. The Ambient Mode start time is measured with [Time to Initial Display](#) (TTID) by ActivityManager.

Python

```
# -df1 to run subatomic_dreamx_from_launcher test.
python3 run_subatomic.py -df1 -o output_dreamx_from_launcher
```

name	unit	description
ambient_from_launcher-start_time	ms	Measures Dreamx start time from launcher

subatomic_katniss_all_during_playback

Measures the client side latency of triggering Google Assistant during video playback.

Google Assistant provides three types of response based on the user's query:

- **Entity:** This redirects the user to a new page.
- **Carousel:** This displays text along with options for the user to choose from, such as recommended YouTube videos.
- **Card:** This provides a simple text response.

This test plays a video and query Google Assistant after the playback has stabilized. The video is played at the device's maximum resolution to adequately stress the system. Each query type will be measured.

Python

```
# -kadb to run subatomic_katniss_all_during_playback test.  
python3 run_subatomic.py -kadb -o output_katniss_all_during_playback
```

name	unit	description
query_assistant_from_hbo-entity_latency	ms	Measures katniss latency during HBO playback
query_assistant_from_hbo-carousel_latency	ms	Measures katniss latency during HBO playback
query_assistant_from_hbo-card_latency	ms	Measures katniss latency during HBO playback

subatomic_katniss_card_during_playback

Measures the latency of triggering Google Assistant during video playback. For Youtube and Netflix, Carousel and Entity queries go through the app, not Google Assistant, so we can only test card queries.

Google Assistant provides three types of response based on the user's query:

- **Entity:** This redirects the user to a new page.
- **Carousel:** This displays text along with options for the user to choose from, such as recommended YouTube videos.
- **Card:** This provides a simple text response.

This test plays a video and query Google Assistant after the playback has stabilized. Video is adjusted to be at max resolution to ensure the system is adequately stressed.

Python

```
# -kcdp to run subatomic_katniss_card_during_playback test.  
python3 run_subatomic.py -kcdp -o output_katniss_card_during_playback
```

name	unit	description
query_assistant_from_netflix2k-card_latency	ms	Measures katniss latency during Netflix playback
query_assistant_from_youtube2k-card_latency	ms	Measures katniss latency during Youtube playback

subatomic_katniss_from_launcher

Measures the latency of triggering Google Assistant from the launcher.

Google Assistant provides three types of response based on the user's query:

- **Entity:** This redirects the user to a new page.
- **Carousel:** This displays text along with options for the user to choose from, such as recommended YouTube videos.
- **Card:** This provides a simple text response.

This test plays video before doing the actual test. This evicts the memory of Google Assistant and the launcher to better simulate the user experience.

This test is much easier on the system than the Google Assistant queries during playback. Even on low end devices this test performs very well.

Python

```
# -kfl to run subatomic_katniss_from_launcher test.  
python3 run_subatomic.py -kfl -o output_katniss_from_launcher
```

name	unit	description
query_assistant_from_launcher-entity_latency	ms	Measures katniss latency from launcher
query_assistant_from_launcher-carousel_latency	ms	Measures katniss latency from launcher
query_assistant_from_launcher-card_latency	ms	Measures katniss latency from launcher

subatomic_launcher_after_playback

Measures the launcher start time and the launcher jank after video playback. This is the most core CUJ in TV and is the focus of many performance evaluations and offers additional performance insights through the disk performance test.

This test has three parts:

1. Measure Launcher start time during playback (Either TTID, and TTFD if cold start)
2. Measure Launcher cold start time (Both TTID and TTFD are measured)
3. Measure disk performance and scheduler metrics during Launcher start

This test plays a video and starts the launcher to measure the start time and jank.

Python

```
# -lap to run subatomic_launcher_after_playback test.  
python3 run_subatomic.py -lap -o output_launcher_after_playback
```

name	unit	description
launcher_from_netflix-launcherx_running_duration	ms	Time launcher spent running on CPU during start
launcher_from_netflix-launcherx_runnable_preempted_duration	ms	Time launcher spent waiting for CPU during start
launcher_from_netflix-launcherx_runnable_duration	ms	Time launcher spent waiting for CPU during start
launcher_from_netflix-launcherx_uninterruptible_sleep_non_io_duration	ms	Time launcher spent waiting for non I/O resources during start
launcher_from_netflix-launcherx_uninterruptible_sleep_io_duration	ms	Time launcher spent waiting on disk during start
launcher_from_netflix-launcherx_sleeping_duration	ms	Time launcher spent sleeping during start
launcher_from_netflix-launcherx_launch_duration	ms	Total time launcher took to display the first frame
launcher_from_netflix-launcherx_unknown_duration	ms	Total time launcher spent in an unknown state during start
launcher_from_youtube-launcherx_running_duration	ms	Time launcher spent running on CPU during start
launcher_from_youtube-launcherx_runnable_preempted_duration	ms	Time launcher spent waiting for CPU during start
launcher_from_youtube-launcherx_runnable_duration	ms	Time launcher spent waiting for CPU during start
launcher_from_youtube-launcherx_uninterruptible_sleep_non_io_duration	ms	Time launcher spent waiting for non I/O resources during start
launcher_from_youtube-launcherx_uninterruptible_sleep_io_duration	ms	Time launcher spent waiting on disk during start
launcher_from_youtube-launcherx_sleeping_duration	ms	Time launcher spent sleeping during start
launcher_from_youtube-launcherx_launch_duration	ms	Total time launcher took to display the first frame
launcher_from_youtube-launcherx_unknown_duration	ms	Total time launcher spent in an unknown state during start

We only show Netflix's data in this **disk statistics** example:

[illegible]

name	metric_type	unit	description
launcher_from_netflix-launcherx_read_ahead_raw_total_size	disk_stat	count	Total time spent of requests being sent to disk and request returned
launcher_from_netflix-launcherx_read_ahead_raw_throughput	disk_stat	count	Throughput of requests being sent to disk and request returned
launcher_from_netflix-launcherx_write_raw_total_duration	disk_stat	count	Total time spent of requests being sent to disk and request returned
launcher_from_netflix-launcherx_write_raw_total_size	disk_stat	count	Total time spent of requests being sent to disk and request returned
launcher_from_netflix-launcherx_write_raw_throughput	disk_stat	count	Throughput of requests being sent to disk and request returned
launcher_from_netflix-read_size	disk_stat	kb	Read request size
launcher_from_netflix-read_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-read_ahead_size	disk_stat	count	Readahead request size
launcher_from_netflix-read_ahead_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-write_size	disk_stat	count	Write request size
launcher_from_netflix-write_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-read_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-read_ahead_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-write_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-read_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-read_ahead_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-write_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-launcherx_read_size	disk_stat	kb	Read request size
launcher_from_netflix-launcherx_read_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-launcherx_read_ahead_size	disk_stat	count	Readahead request size
launcher_from_netflix-launcherx_read_ahead_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-launcherx_write_size	disk_stat	count	Write request size
launcher_from_netflix-launcherx_write_queuing_latency	disk_stat	count	Latency between the request queued in block layer and sent to disk
launcher_from_netflix-launcherx_read_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-launcherx_read_ahead_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-launcherx_write_raw_latency	disk_stat	count	Latency between the request sent to disk and request being completed
launcher_from_netflix-launcherx_read_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-launcherx_read_ahead_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-launcherx_write_latency	disk_stat	count	Latency between the request queued in block layer and request being completed
launcher_from_netflix-launcherx_interleave_count	disk_stat	count	Number of interrupting requests queued and completed between the request being queued in

subatomic_memory_breakdown

Measures the kernel, userspace, device, and carveout memory during the core TV CUJS for:

- Youtube
- Netflix
- Exoplayer playback
- Launcher

It also measures userspace process memory for a select number of processes.

This test also categorizes memory into buckets: 1st party app usage, 3rd party app usage, OS usage, and vendor process usage. This helps to visualize the memory usage at each software layer on the system.

In addition, it also provides process memory usage summaries for each process in the system.

The **userspace process memory usage** is measured as: $PSS + (PSS \text{ Swap} * \text{Compression Ratio})$

The **memory usage** is measured as the real RAM cost:

- For kernel, device, and carveout usage, this is RAM used by each component.
- For userspace processes, e.g. apps, the real RAM usage is more complicated due to swapping and shared pages.

Python

-mem_all to run all subatomic_memory_breakdown test.

```
python3 run_subatomic.py -mem_all -o output_memory_breakdown_all
```

-mem_yt to run youtube subatomic_memory_breakdown test.

```
python3 run_subatomic.py -mem_yt -o output_memory_breakdown_youtube
```

-mem_nflx to run netflix subatomic_memory_breakdown test.

```
python3 run_subatomic.py -mem_nflx -o output_memory_breakdown_netflix
```

-mem_exo to run exoplayer subatomic_memory_breakdown test.

```
python3 run_subatomic.py -mem_exo -o output_memory_breakdown_exoplayer
```

-mem_lx to run launcher subatomic_memory_breakdown test.

```
python3 run_subatomic.py -mem_lx -o output_memory_breakdown_launcher
```

We only show Netflix's data in this example:

name	unit	description
netflix_playback-systemui_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-settings_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-vending_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-mediashell_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-launcherx_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-launcherx_coreservices_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-gms_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-gms_persistent_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-katniss_interactor_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-netflix_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-surfaceflinger_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-system_server_pss	mb	Process total RAM consumption as PSS + PSS Swap compressed memory
netflix_playback-carveout	mb	Memory reserved at boot time for kernel and hardware
netflix_playback-hardware	mb	Estimated runtime hardware memory usage
netflix_playback-kernel	mb	Total runtime kernel memory usage
netflix_playback-userspace	mb	Total amount of memory available to userspace processes
netflix_playback-vendor_process_pss	mb	Total RAM used by vendor processes
netflix_playback-os_pss	mb	Total RAM used by the OS
netflix_playback-apps_1p_pss	mb	Total RAM used by 1p Apps
netflix_playback-apps_3p_pss	mb	Total RAM used by 3p Apps
netflix_playback-kernel_slab_unrec	mb	Kernel slab unreclaimable memory usage
netflix_playback-kernel_slab_rec	mb	Kernel slab reclaimable memory usage
netflix_playback-kernel_vmalloc	mb	Kernel vmalloc memory usage
netflix_playback-kernel_stack	mb	Kernel stack memory usage
netflix_playback-kernel_page_tables	mb	Kernel page table memory usage
netflix_playback-kernel_shmem	mb	Kernel shmem memory usage
netflix_playback-kernel_percpu	mb	Kernel percpu memory usage
netflix_playback-kernel_buffers	mb	Kernel buffer memory usage
netflix_playback-kernel_used	mb	Total amount of kernel uncached memory
netflix_playback-kernel_cached	mb	Total amount of kernel cached memory
netflix_playback-mapped	mb	Total amount of userspace mapped memory
netflix_playback-anonymous	mb	Total amount of userspace anonymous memory
netflix_playback-mem_avail	mb	Amount of memory available as reported by MemAvailable
netflix_playback-zram_comp	mb	Total amount of compressed memory used for zram swap
netflix_playback-unmapped	mb	Total amount of userspace unmapped memory
netflix_playback-free	mb	Total amount of userspace free memory

subatomic_memory_stats

Measure memory stats for the Settings overlay over:

- app switching
- video playback (Youtube and Netflix)

which are key performance CUIs.

This test is used to measure the memory performance of a device, i.e. how well the device is able to handle memory pressure. Its main use case is to evaluate how well the VM, ZRAM, and LMK settings of a device are tuned. It can also highlight issues with disk reads and compaction.

Python

```
# -stat_app to run subatomic_memory_stats app switch test.
python3 run_subatomic.py -stat_app -o output_memory_stats_app_switch

# -stat_play to run subatomic_memory_stats playback test.
python3 run_subatomic.py -stat_play -o output_memory_stats_playback
```

We only show the app switch data in this example:

name	unit	description
app_switch-ws_refault_total	count	Total amount of working set page refaults
app_switch-pages_paged_in	count	Total amount of pages paged into RAM
app_switch-allocation_stalls	count	Total amount of direct allocation stalls
app_switch-major_pagefaults	count	Total amount of major page faults
app_switch-kswapd_page_steals	count	Total amount of pages stolen by kswapd
app_switch-direct_page_steals	count	Total amount of pages stolen during allocation stalls
app_switch-kswapd_page_scans	count	Total amount of pages scanned by kswapd
app_switch-direct_page_scans	count	Total amount of pages scanned during direct allocation stalls
app_switch-kswapd_low_wm_hit_quick	count	Total times the low watermark was rebreached too quickly
app_switch-compaction_stalls	count	Total amount of compaction stalls
app_switch-mmc_reads	count	Total amount of MMC reads
app_switch-mmc_reads_ms	count	Total time spent reading the MMC
app_switch-zram_reads	count	Total time spent reading from ZRAM
app_switch-psi_io_some_total_ms	ms	Pressure Stall Information for I/O in ms
app_switch-psi_io_full_total_ms	ms	Pressure Stall Information for I/O
app_switch-psi_mem_some_total_ms	ms	Pressure Stall Information for memory
app_switch-psi_mem_full_total_ms	ms	Pressure Stall Information for memory
app_switch-psi_cpu_some_total_ms	ms	Pressure Stall Information for CPU

subatomic_settings_during_playback

Measures the latency of opening Settings overlay when the user presses the settings button during video playback.

This test plays video, sets up video, then opens Settings overlay. The video is played at the device's maximum resolution to adequately stress the system.

Python

```
# -sdp to run subatomic_settings_during_playback test.  
python3 run_subatomic.py -sdp -o output_settings_during_playback
```

name	unit	description
settings_overlay_from_netflix2k-start_time	ms	Measures settings dashboard latency during Netflix playback
settings_overlay_from_youtube2k-start_time	ms	Measures settings dashboard latency during Youtube playback

subatomic_settings_from_launcher

Measures the latency of opening settings overlay and settings app from the launcher.

Measures:

- Settings overlay latency (Settings button keypress)
- Settings app latency (pressing Settings icon in Settings overlay)
- Settings jank from scrolling the Settings app

This test first plays video before returning to the launcher. This clears the memory of the launcher and Settings. It then opens the Settings overlay (press Settings button) and the Settings app. After that, the test scrolls the Settings app to test jank.

Python

```
# -sf1 to run subatomic_settings_from_launcher test.  
python3 run_subatomic.py -sf1 -o output_settings_from_launcher
```


name	unit	description
settings_overlay_from_launcher-start_time	ms	Measures settings dashboard latency from launcher
settings_app_from_launcher-start_time	ms	Measures settings app latency from launcher
settings_app_from_launcher-settings_jank	ms	Measures settings app jank

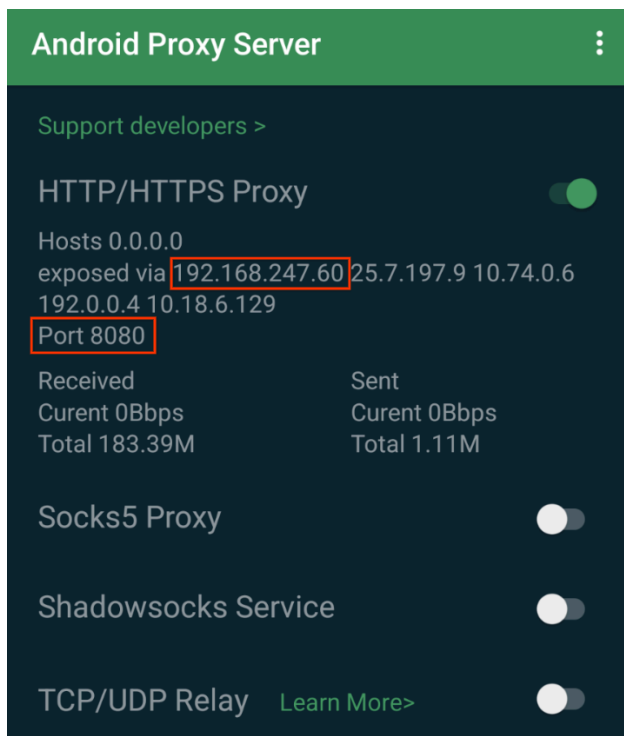
Appendix: Mobile VPN hotspot for TV

Directly doing the VPN on TV could affect the performance result. Therefore, we recommend using a mobile VPN hotspot, and connect the TV to the mobile VPN.

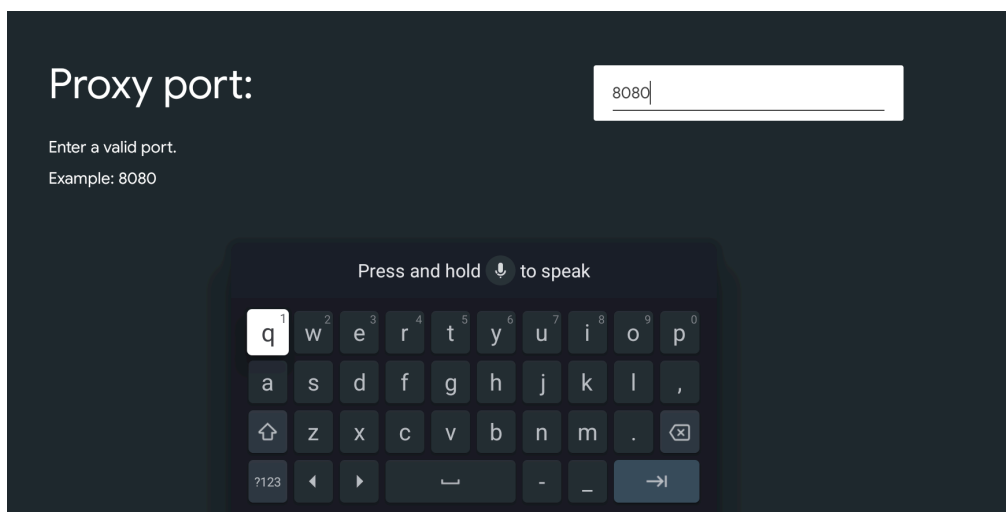
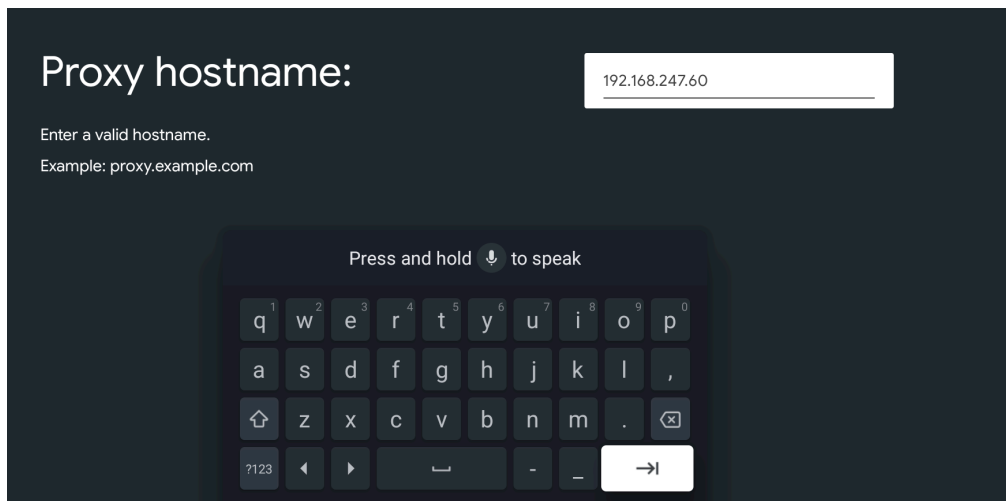
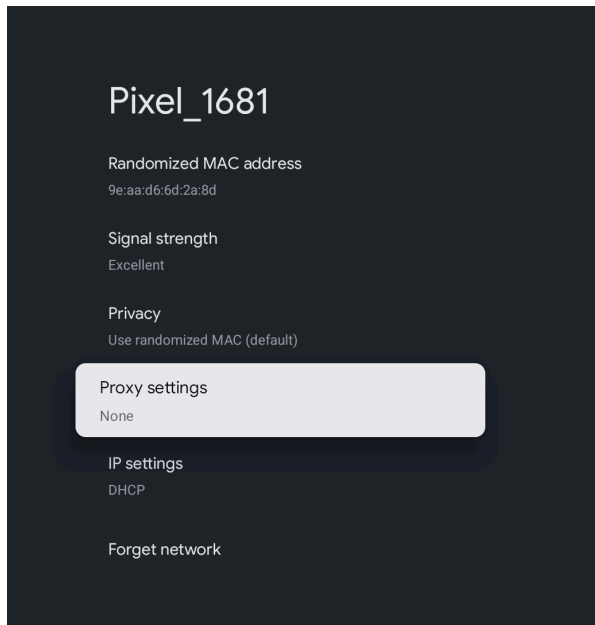
1. Download Android proxy server app on mobile: [Android Proxy Server - Apps on Google Play](#)
2. Connect to US VPN on mobile.
3. Enable hotspot on mobile (**must do this before next step**).
4. Open Android proxy server app, and enable HTTP/HTTPS Proxy.

Host: **192.168.247.60**

Port: **8080**



5. Connect TV to mobile hotspot.
6. After connecting, follow the steps below to setup wifi proxy:



No need to change default settings for Bypass proxy:

Bypass proxy for:

example.com,mycomp.test.com,localhost

Enter a comma-separated list of excluded domains or leave blank.

Example: example.com,mycomp.test.com,localhost

localhost

